

DESARROLLO ORIENTADO A OBJETOS [DOPO]

2026-2

Ciclo 4

Materia: Desarrollo orientado a objetos

Grupo: 7

Integrantes

Jhon Eduard Duran Ceballos

ID: 1000100325

Correo: jhon.duran-c@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez

ID: 1000109048

Correo: andres.oyuela-m@mail.escuelaing.edu.co

MINI-CICLO 1: Extensión y Polimorfismo de las Ruedas

Estado: Completado.

Objetivo: Implementar herencia en la clase Wheel para soportar mínimo tres tipos de comportamiento de ruedas.

Descripción: Se refactorizó la gestión de ruedas para permitir instanciar ruedas con comportamientos anómalos, asegurando que cada una se distinga visualmente en el lienzo.

- Implementación de la clase LeftyWheel (hereda de Wheel), modificando su borde a color morado y sobrescribiendo el método de giro para copiar el estado de la rueda a su izquierda.
- Implementación de la clase RebelWheel, adaptándola para rechazar acciones de bloqueo (lock), intercambio (swap) o eliminación (delWheel).
- Actualización de SlotMachine para manejar estos nuevos tipos polimórficos de ruedas.

MINI-CICLO 2: Extensión de los Símbolos y Patrón Prototype

Estado: Completado.

Objetivo: Soportar nuevos tipos de símbolos dinámicos (ephemeral y shy) evitando conflictos de renderizado en la librería gráfica.

Descripción: Se transformó la clase Symbol para que funcione bajo el Patrón Prototype, permitiendo clonar copias de los objetos en lugar de pasar cadenas de texto, lo que habilitó a los símbolos para guardar y modificar su propio estado en cada giro.

- Inserción de los métodos cambiarTamano(), aplicarEfecto() y generarCopia() en la superclase Symbol.
- Creación de la clase EphemeralSymbol, programada para reducir progresivamente su tamaño en cada giro.
- Creación de la clase ShySymbol, que alterna su estado de visibilidad cada vez que es seleccionada.
- Refactorización de los métodos spin() y placeSymbol() en SlotMachine para enviar clones de objetos en lugar de simples variables tipo String.

MINI-CICLO 3: Creación de un Elemento Inédito

Estado: Completado.

Objetivo: Cumplir con el requisito creativo de ofrecer un nuevo tipo de elemento para el simulador.

Descripción: Se diseñó e implementó la LazyWheel (La Rueda Perezosa). Esta rueda representa una mecánica de casino en la que la máquina está "oxidada" o "pesada", por lo que al momento de girar tiene un 60% de probabilidad de hacerlo normalmente y un 40% de probabilidad de abortar el giro, reteniendo el símbolo que ya tenía.

- Creación de la clase LazyWheel que hereda de la superclase Wheel.
- Sobrescritura polimórfica del método girar(Symbol nuevoSimbolo) para incluir una condicional Math.random() >= 0.4; si se cumple, delega a super.girar(), de lo contrario, elude la orden y conserva el símbolo.
- Creación del método addLazyWheel(int pos) en la clase SlotMachine para inyectar la rueda.
- Implementación de una cascada de validaciones defensivas en addLazyWheel para verificar los límites de inserción y consultar si la posición estaba ocupada por una rueda bloqueada (isLocked()).

MINI-CICLO 4: Pruebas, Diseño UML y Cierre de Proyecto

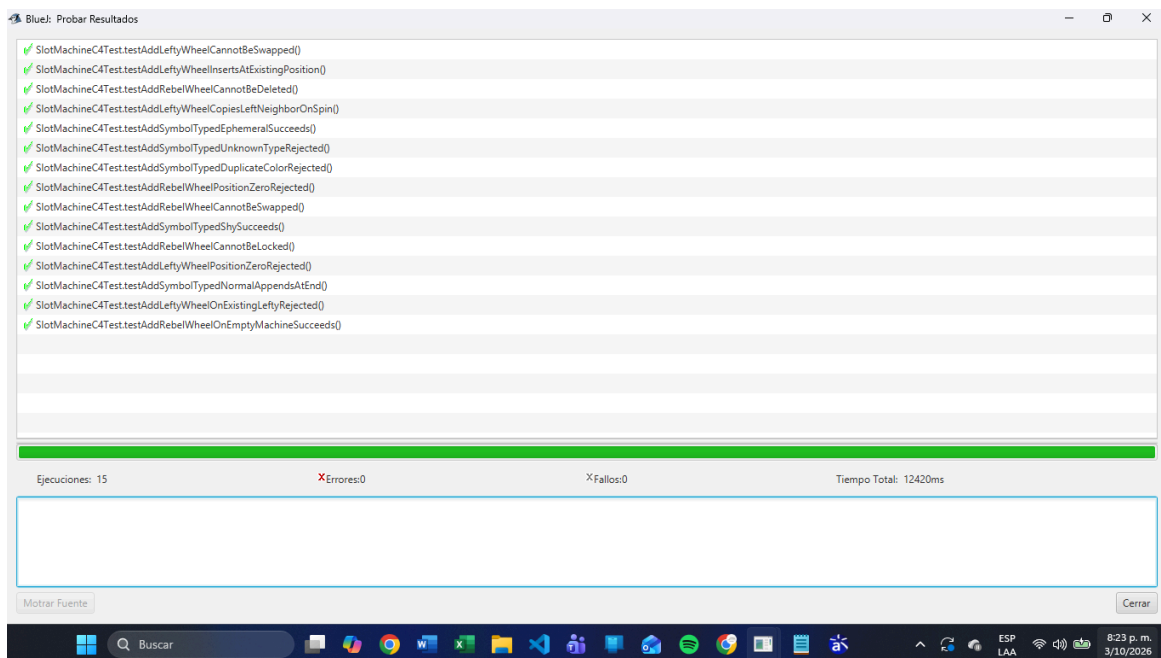
Estado: Completado.

Objetivo: Validar la estabilidad del sistema polimórfico y consolidar los entregables finales del curso.

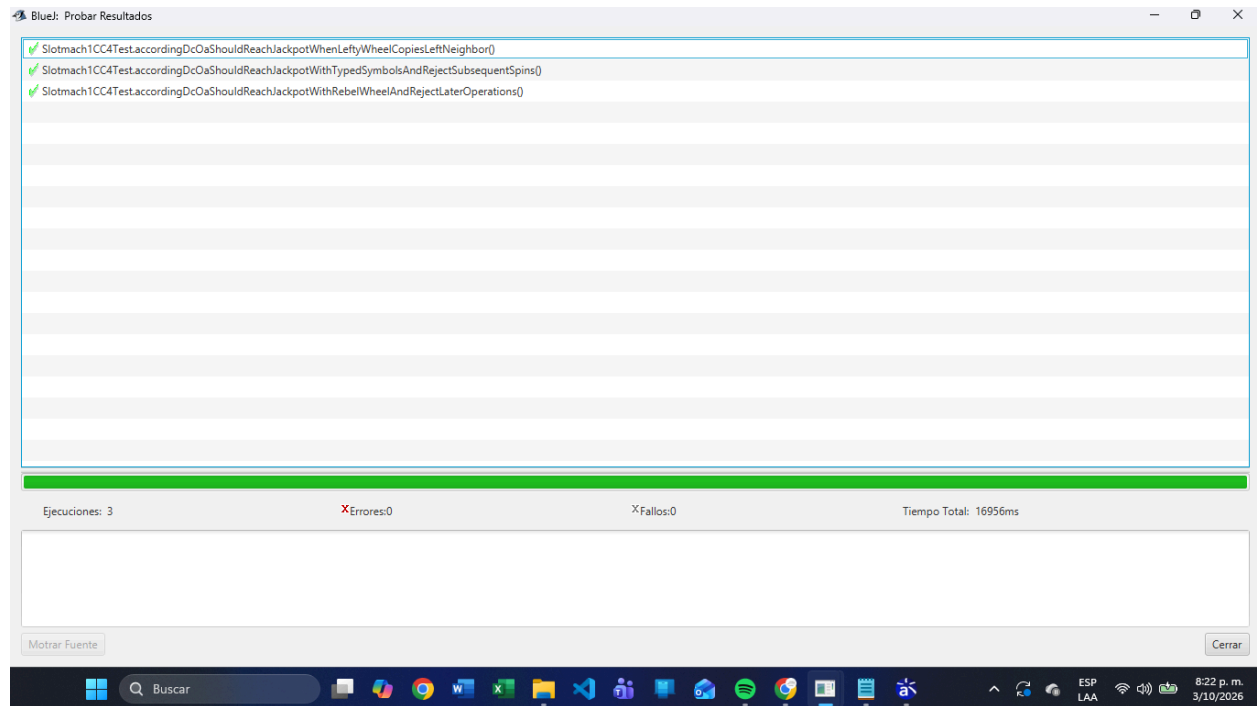
Descripción: Construcción de las baterías de pruebas automatizadas en modo invisible, actualización de la arquitectura en Astah y redacción de este documento.

- Creación de la clase de pruebas unitarias SlotMachineC4Test.
- Codificación de casos compartidos en SlotMachineCC4Test.
- Actualización del diagrama de clases en Astah para reflejar la herencia.
- Validación general de estándares de documentación JavaDoc.

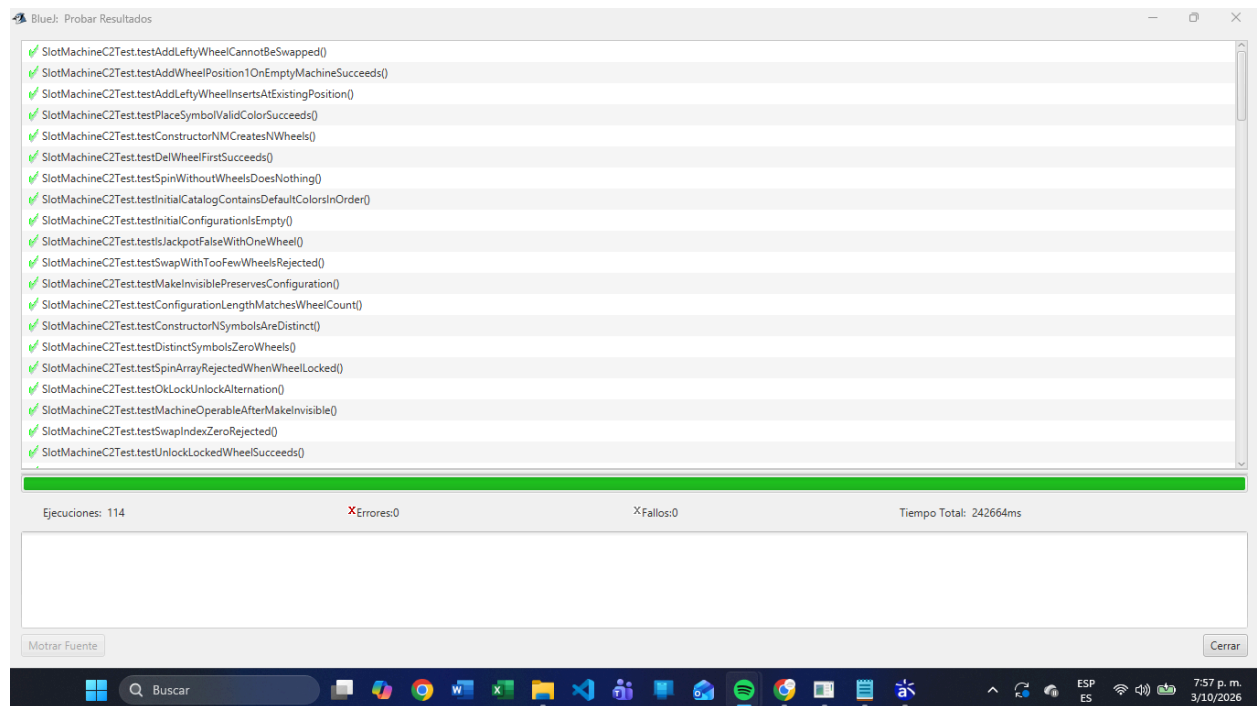
Captura de pantalla (Pruebas para SlotMachineC4Test)



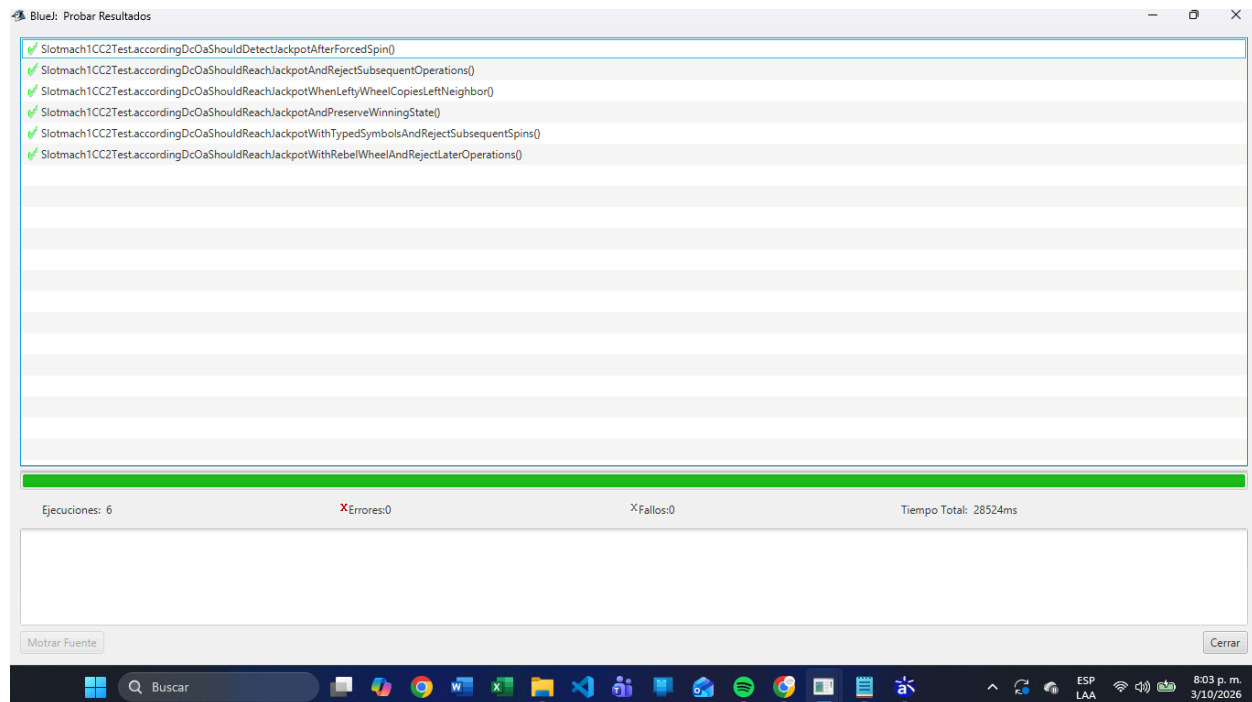
Captura de pantalla (Pruebas para SlotMachineCC4Test)



Captura de pantalla (Pruebas generales de atributos para slotMahcine)



Captura de pantalla (Caos de pruebas de unidad compartidos para slotMachine)



2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

El estado actual es 100% completado. Logramos abarcar la totalidad de los requisitos del cuarto ciclo. El simulador es ahora completamente extensible; maneja exitosamente las clases LeftyWheel, RebelWheel, EphemeralSymbol y ShySymbol, y todas las funcionalidades fueron validadas superando satisfactoriamente las pruebas de unidad y de aceptación exigidas sin afectar la funcionalidad de la maratón de los ciclos anteriores.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Invertimos aproximadamente 10 a 12 horas/hombre cada uno.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue la exitosa implementación del Patrón Prototype en la gestión de símbolos. Logramos transformar la arquitectura para que el catálogo de la máquina entregara copias exactas (generarCopia()) de los símbolos a las ruedas. Esto no sólo habilitó el polimorfismo para que los símbolos pudieran cambiar de tamaño o visibilidad por sí solos, sino que solucionó definitivamente el "bug de teletransportación" en el Canvas de BlueJ.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- Problema: Al intentar pasar a una arquitectura polimórfica, el compilador lanzaba errores de compatibilidad (method girar cannot be applied to given types), ya que métodos

antiguos de la máquina como `placeSymbol` o `spin` estaban programados rígidamente para enviar textos (`String color`, `String forma`) en lugar de objetos completos.

- Solución: Tuvimos que realizar un "Refactoring" profundo en la clase `SlotMachine`. Reemplazamos la lógica de búsqueda por una iteración que encontraba el objeto `Symbol` en el catálogo original, ejecutaba su método polimórfico `aplicarEfecto()` y finalmente despachaba su clon hacia la rueda correspondiente, manteniendo la cohesión entre la memoria del programa y la interfaz gráfica.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Logramos una excelente planificación. En lugar de intentar meter el código de las nuevas ruedas y símbolos a la fuerza, decidimos detenernos, analizar el código heredado y preparar las superclases (`Wheel` y `Symbol`) abriéndolas a la extensión mediante herencia. Esta comunicación nos permitió dividir el trabajo sin pisarnos el código.

Nos comprometemos a aplicar el principio de "Diseño Abierto/Cerrado" (Open/Closed Principle) desde las etapas más tempranas en futuros proyectos de software. Si hubiéramos concebido los símbolos como Objetos desde el Ciclo 1 en lugar de tratarlos como simples textos (`Strings`), nos habríamos ahorrado gran parte del tiempo invertido en la refactorización de este último ciclo.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica más útil en esta entrega fue la de Refactoring (Refactorización) combinada con Small Releases (Entregas Pequeñas). Al aislar la refactorización de la superclase `Symbol` en un solo paso antes de crear a `EphemeralSymbol` o `ShySymbol`, garantizamos que la base estuviera sólida. De no ser por la refactorización iterativa, la complejidad de mezclar herencia con la librería gráfica habría vuelto el código inmanejable.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No. 4/4 — Refactoring y Extensión*. Documento de requisitos, restricciones de diseño y diagrama UML.
- ICPC. (2025). *Problem I: Slot Machine*. International Collegiate Programming Contest.
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *DOPO - Instructivo de Pruebas Unitarias y Colectivas*. Guía para la convención de nomenclatura (accordingDcOmShould...).
- JUnit Team. (2025). *JUnit 4 Documentation*. Framework para pruebas unitarias automatizadas.